Split any bill onchain. Let the agents do the work.

Programmable Money Hackathon

Agentic Economy track · DeFi track

splitsy.xyz · github.com/qFloppa/Splitsy

The problem

Splitting a bill still ends in a group chat full of IOUs. "I'll send it later," and then nobody does.

- 🔐 Crypto-native splitting exists, but it assumes **everyone already holds a wallet and a gas token**.
- 📝 Debts live in someone's notes app, so there is **no shared record and no consequence** for never paying.
- 🧮 The receipt work itself, scanning and parsing and currency conversion, is **labour nobody accounts for**.
- 🧾 And chasing the money is a person's job. Somebody has to remember, and ask, and ask again.

> Splitsy makes the debt real, makes joining frictionless, makes the scanning pay for itself, and hands the chasing to an agent.

🧾 What Splitsy does

Snap a receipt, split it, everyone pays their share in USDC on Arc.

📸 Scan

An agent buys the parse: merchant, line items, tax, tip, currency.

👥 Split

Assign shares, including to people who hold no wallet yet.

🔗 Commit

Every debt is escrowed onchain in `BillSplitRegistry`.

💸 Settle

Real USDC moves, the creator claims, reputation is written onchain.

Splitsy never takes custody. The contract holds the funds and the truth. Up to 256 participants per bill.

⚡ Agentic Economy

Five agents, seven paid endpoints, one live market

🤖 Scout: upload a receipt, an agent goes shopping

Scout is an autonomous agent with a real economy of its own.

- 🪪 Its own **wallet and ERC-8004 identity** on Arc.
- 💰 A **daily budget it will not exceed**, enforced in atomic base units.
- 🔍 It **judges your photo before spending anything.**
- 🛒 It **pays Splitsy's own x402 endpoints per call** in USDC fractions.
- 🔁 It **buys a second opinion** when the parse looks shaky.

> Splitsy is both the buyer's principal and the seller. It is a closed loop you can watch settle live on chain.

💸 x402 nanopayments on Arc

1. The buyer POSTs. The seller answers `402 Payment Required` with a base64 `PAYMENT-REQUIRED` challenge naming scheme, network, asset, and amount.
2. The buyer signs a **gasless EIP-3009 authorization**. No transaction, no gas token.
3. Circle Gateway is the facilitator: it runs `verify()` then `settle()`, **batching** the payment on Arc.
4. `200 OK` with a `PAYMENT-RESPONSE` transaction hash, ledgered as **earned** on one side and **spent** on the other.

`maxTimeoutSeconds` is not hardcoded. The seller reads Arc's own `minValiditySeconds` from the facilitator and adds an hour of margin, because Gateway checks validity remaining at verification time, not at signing time.

🧠 Scout's decision logic

Pure functions, fully unit-tested, because this is the logic that spends money.

- 🔍 **Assess first.** Reject anything under **8 KB** or **200 px** on either edge **before paying anything**.
- 💰 **Budget gate.** Spend checks run in atomic base units, never floats, so the cap holds exactly at the boundary.
- 🔁 **Second opinion.** If confidence is **below 0.80 and** the budget allows, pay again for a high-rigour re-scan and keep the better parse.
- 🌍 **Foreign currency.** Pay the FX seller only when the bill is not already in USD.
- 🛟 **Graceful degradation.** If the paid path fails, fall back to a direct parse. **The human upload never breaks.**

📊 One scan, one live agent economy

3
nanopayments

$0.011
total spend

0
gas tokens touched

Call	Paid	Result
`/api/ocr`	$0.005	confidence 0.62, below the gate ⚠️
`/api/ocr` `{hq:true}`	$0.005	confidence 0.94, kept ✅
`/api/fx`	$0.001	EUR is not USD 🌍

The dashboard's agent-economy panel tracks earned against spent, calls served against calls paid, and budget remaining, updating as Scout works.

🏛️ Five agents, and none of them grades itself

Scout was the start. Autopay is now a **three-party ERC-8183 job** on Arc's already-deployed `AgenticCommerce` contract, plus a creditor-side sweeper.

Scout

Buys the receipt parse. Server-held EOA, its own identity.

Your agent

One per account. Posts the job, escrows the fee, pays the share.

The Settler

Provider. A raw EOA, because x402 needs a real key to sign EIP-3009.

The Auditor

Evaluator. It is **paid to say no**, and it reads the chain to do it.

Plus the Validator, which signs every ERC-8004 `giveFeedback`, and a registrar that holds payers' identity NFTs only long enough to hand them over. No new Solidity: every contract here was already deployed.

🔐 The job ceremony

```
0. decide     lib/autopay.ts rules, then a bill review BOUGHT from the
              Auditor over x402. Any refusal stops here: no job, 0 tx.
1. createJob  your agent          <- client
2. setBudget  the Settler         <- the provider prices its own work
3. fund       your agent          -> the fee into escrow
4. settle     payDebtFor(billId, debtor, amount)
5. submit     the Settler         -> keccak256(settlementTxHash)
6. complete   the Auditor, ONLY after reading getParticipant on chain
              and seeing paid >= owed
```

Step 6 is not a rubber stamp. If the debt is not really settled the Auditor does not complete, the job expires after an hour, and the Settler is not paid for work it did not do.

⚖️ What that design actually buys

Six transactions per settled **share**, not per bill. And the honest ledger of when you pay them:

Event	Extra transactions
bill created, or paid by hand	**0**
autopay off	**0**
autopay on, the agent **refuses**	**0**
autopay on, the agent **pays**	**6**

- 🔒 **The escrow only ever holds the fee.** Bill money is never in it, so a broken settlement strands cents, not a share.
- 💵 **The agent's balance is the hard ceiling.** Funding is a transfer, not an approval: an agent holding 5 USDC can never spend 6.
- 🧾 `payDebtFor` emits `DebtPaid` naming the **debtor** as payer, so reputation flows to the **user**, never to their agent.

💳 The review the Settler has to buy

`autopay-review` used to be a free internal function call. It is now a service the **Auditor sells at $0.002** and the Settler buys over x402 out of its own fee income.

- 📒 Both sides land in the `x402_payments` ledger: **earned** by the seller, **spent** by the buyer.
- ⏱️ The `spent` row is written **before** the body is inspected, because by then Gateway has already settled. An unreadable verdict still costs the fee.
- 🚫 Every failure direction is a refusal: a 402, a timeout, an unparseable verdict, a missing key.

> **A Settler that cannot buy a review settles nothing.** The fee is priced under the job fee, so the Settler stays ahead on a job it completes.

🛒 The marketplace: seven endpoints anyone can buy

Splitsy does not just consume x402, it **sells on it**. Every one of these is public to whoever pays.

Endpoint	Price	What you buy
`/api/ocr`	**$0.005**	A receipt parsed
`/api/fx`	**$0.001**	A currency converted
`/api/agents/review`	**$0.002**	A verdict on whether a bill holds up
`/api/agents/queue`	**$0.001**	The bills a wallet's mandate would pay right now
`/api/agents/netting`	**$0.001**	A minimal-transfer settlement plan
`/api/agents/dunning/verdict`	**$0.001**	Nudge, escalate, collect, or do nothing
`/api/reputation`	**$0.001**	An ERC-8004 payment score for underwriting

`/api/agents/catalog` is free: it is discovery, not a service. `/api/agents/skill` serves a Circle-shaped skill file so a stranger's Circle Agent Wallet can learn the autopay flow, with the live mandate address templated in.

🧹 The creditor-side agent: dunning as a daily sweep

Not every debt waits on the debtor. The **dunning agent** sweeps once a day, checking every bill a Splitsy user created that carries a due date and still has an unpaid share.

Situation	Action	Reach
Not yet due, but **within 3 days**	nudge	Email only
Overdue, **mandate + funds**	collect	`collectDebt(billId, debtor, collectible)`
Overdue, **no mandate or no funds**	escalate	Email only

- 🔁 Runs **daily at 12:00 UTC** (Vercel cron).
- 🔐 Signed by the creditor's Circle wallet. No new custody.
- ✉️ Email is the only channel wired up. X and Discord scopes are sign-in only.

🏛 DeFi

The net-settlement treasury

⚖️ Many debts. One position.

Real groups owe each other across dozens of bills. The dashboard collapses all of it into **one net figure per counterparty**, sorted by largest exposure.

Counterparty	Owed to me	I owe	Net
`@dev`	$0.00	$43.75	**−$43.75**
`@carla`	$18.40	$0.00	**+$18.40**
`sam@example.com`	$7.00	$0.00	**+$7.00**

Counterparties resolve to live handles rather than raw hex. Both directions net into a single row per person.

⚡ Settle net atomically — from either wallet

Settling bill by bill costs an approve plus a `payDebt` per debt, plus one `claim` each. Five debts and four claims is `2 × 5 + 4`.

14 transactions → 1

The registry's own `settle(claimIds, payIds, amounts)` carries **every leg in one call**, running the claims before the pays so their proceeds fund the payments inside the same transaction. All-or-nothing: one bad leg reverts the batch.

- 🟣 **Circle SCA wallet:** `executeBatch` wraps the approval and the `settle` into **one** atomic transaction.
- 🦊 **Browser EOA:** the same `settle`, so **two** prompts — one approval, one settle — whatever the leg count. One when there is nothing to pay.

> Escrow binds each debt to its `billId`, so batching removes **transactions, not transfers**. The net figure is exposure. We do not claim less USDC moves.

🔧 Why the treasury is real infrastructure

- 🧮 **Pure aggregation core.** Registry reads in, net positions out. All money math in base-unit `bigint`, one formatter at the wire boundary, and `bigint` never crosses `Response.json`.
- 🔒 **Chain-verified execution.** Both settle paths **re-read every outstanding amount from chain before signing**. A stale dashboard can never sign a wrong amount, and neither path accepts a client-supplied figure.
- ⚛️ **Atomicity in the contract, not the wallet.** `settle` is bounded at 64 legs per array and reverts as a unit, so a browser EOA gets the same all-or-nothing guarantee an SCA `executeBatch` gives — no half-settled state to unwind on either path.
- ✅ **Tested.** The aggregation core and the `executeBatch` encoder both carry unit tests, including a round-trip decode of every leg in order.

👥 Anyone can be in a split

Sign in with **X, Discord, Google, or email**. Splitsy provisions a Circle smart-contract wallet on Arc behind the login. No seed phrase, no extension.

📨 Tag someone with no account at all

Their share sits in escrow on Arc until they claim it.

🔀 Mix social identities and browser wallets

An X handle, an email, and a `0x` address can all owe on the same bill.

🖋️ The creator picks their identity

Bill creation from the Circle wallet or a connected browser wallet.

💬 One-sentence debts

The IOU composer: "@dani owes me $42" → registry bill. "I owe @dani $42" → direct transfer.

🌉 Bring USDC from wherever it already is

Someone's USDC is rarely on the chain you need. Splitsy pulls it in on **two Circle rails**, both native, neither wrapped.

CCTP v2, burn-and-mint through App Kit, in the settle deck.

Base Sepolia · Ethereum Sepolia · Arbitrum Sepolia · Optimism Sepolia · Avalanche Fuji · Polygon Amoy

Circle Gateway, on the public pay link, browser wallet, fully client-side.

Avalanche Fuji · Base Sepolia · Ethereum Sepolia → Arc (domain 26)

- ✍️ Gateway is **two signatures, no server keys**: an EIP-712 `BurnIntent` on the source chain, then `gatewayMint` on Arc.
- ⛽ A **Paymaster** path covers source-chain gas in USDC when the native balance is too low to bridge, via an EIP-2612 permit run as an EIP-7702 authorization.

🔍 Verifiable, escrowed debt

Every bill commits a **metadata hash** onchain at creation.

- 🧾 Before paying, the payer's client **recomputes that hash from the off-chain preimage**. Merchant, total, and their own share must match what Arc records, or the UI warns them not to pay.
- 🔐 `payDebt` escrows USDC **per** `billId`. The creator calls `claim` to collect.
- 🎯 **All-or-nothing bills.** Opt into `escrowUntilFull` and claims are withheld until every participant has paid. If the bill is still short at its deadline, payers take their own money back with `refund`: self-service, no admin key, nothing strandable.
- 📡 A **Circle Smart Contract Platform event monitor** watches `DebtPaid`, so payments sent straight from a browser wallet, which never touch Splitsy's servers, still fire webhooks and still earn reputation.

Trust-minimised by construction: the app cannot move, redirect, or withhold anyone's money.

🔗 One link. Anyone pays. No account.

Flip **"Anyone can pay"** at creation and the bill mints a public share link. A stranger can open it and cover **any** payer's share.

22 chars · ~131 bits

- 🎲 The token **is** the access control, so it is drawn from a rejection-sampled base62 alphabet: every character uniform, none likelier than the rest.
- 🚫 Deliberately **not the bill id**. Ids are sequential, so a link built from one would let anyone walk the entire registry by counting.
- 💳 Pay from a Splitsy wallet, a browser wallet, or **cross-chain via Gateway**, right from the link.
- 🔒 The escrow badge on the page says plainly whether funds are locked or the bill is complete.

📅 "Pay by": a deadline, committed up front

The creator can set an **optional** due date. Leaving it blank keeps the bill exactly as it would be if due dates did not exist.

The date is **hashed into the bill's onchain metadata at creation**, which is the whole point.

- 🚫 The creator **cannot move it, add one, or backdate it** afterwards. Doing so breaks the hash the payer verifies.
- ⛓️ The contract refuses a due date already in the past, so a bill can never be created instantly collectible.
- 👀 Payers see the deadline **before** they pay, framed as what it is: paying on time keeps your reputation strong.

A committed deadline is a promise the creator makes too, not a lever they hold.

🛡️ Why "Pay by" cannot grief anyone

Worst case: 50 / 100

Situation	Score	Tag
No deadline set	**100**	`paid_in_full`
Within due date **+ 2-day grace**	**100**	`paid_on_time`
Past grace	100 less 5 per day, **floor 50**	`paid_late`

- ➕ **Paying is always positive.** Even very late, a completed payment is good faith, so the floor is a passing 50. No zero, no negative.
- 🕓 A **two-day grace window** absorbs timezone slack and "paid the morning it was due".
- 🤖 **A pull is never scored.** Being auto-debited at a deadline is not the same signal as choosing to pay on time, so dunning collections record nothing about intent.

🏅 ERC-8004 payment reputation

Portable, verifiable reputation on the ERC-8004 registries Arc pre-deploys.

- 🪪 **The payer owns their identity NFT.** The registrar mints it, then transfers it to the payer.
- 🚫 ERC-8004 forbids scoring your own agent, so **the Validator scores, never the payer**. A `(wallet, bill_id)` claim makes double-minting and double-scoring impossible.
- 🔗 Each score commits `keccak256("splitsy:bill:<id>:<payTx>")`, so **anyone can recompute a score against the exact payment it grades**.
- ⛓️ Timing is graded on the `payDebt` **block timestamp**, not a server clock, identically across the Circle wallet, webhook, and replay paths.
- ⚖️ The badge aggregate is **amount-weighted by share**, so a large late bill matters more than a small one.

Lose the database and the chain is still the audit trail: a replay script re-derives every score through the same pure curve, idempotent per (payer, bill).

🔄 Recurring tabs: subscriptions on Arc

For weekly shared bills and monthly services, Splitsy deploys a **recurring tab contract** with a recipient, a cycle length, member wallets, and fixed USDC shares.

Allowance-based, not prepaid.

1. ✍️ Each payer approves the tab contract **once**, for a USDC limit they choose.
2. 🏦 **Funds stay in the payers' own wallets** until a cycle is actually due.
3. ⏰ A backend settler calls `settleTab()` on a **daily cron**. The contract pulls the fixed share from every member with enough balance and allowance, **skips the rest**, and makes the collected USDC claimable by the recipient.
4. 🔓 A payer **revokes at any time** by setting the tab allowance back to `0`.

Each cycle is scored independently for reputation, keyed `tab:<id>:cycle:<n>` and graded against that cycle's boundary. Recurring tabs accept the same mixed social-and-wallet membership as one-off bills.

🧱 The stack, with receipts

Layer	What it is
Arc Testnet	chainId `5042002`, sub-second finality, Malachite BFT
USDC-native gas	`0x3600…0000`. A share and its fee are one asset
Circle Wallets	`accountType: "SCA"` on `ARC-TESTNET`, per login
ERC-4337 + ERC-1967	proxy over `circle_6900_singleowner_v3`
CCTP v2	native burn-and-mint from 6 testnet chains
Circle Gateway	`GatewayMinter 0x0022222A…`, 3 chains → Arc domain 26
Paymaster v0.8	`0x3BA9…8966`, source-chain gas paid in USDC
x402	`@circle-fin/x402-batching`, Gateway as facilitator
ERC-8004	`IdentityRegistry 0x8004A818…` + ReputationRegistry
ERC-8183	`AgenticCommerce 0x0747EEf0…`, already deployed
Circle SCP	`DebtPaid` event monitor to webhooks

Next.js 16.2 · React 19.2 · viem 2.52 · Hardhat 3 · Supabase · Multicall3 batched reads

🔒 Contracts written like they hold money

Contracts of our own, and no external Solidity dependencies at all.

`BillSplitRegistry`

Bills, partial payments, escrow, refunds, claims, per-bill collect mandates.

`RecurringTab` + factory

Allowance-pull subscriptions, one contract per tab.

Shared primitives

Our own `ReentrancyGuard` on every fund-moving entrypoint, and a reverting `SafeERC20`.

Hardhat 3 tests plus a Slither config in the repo. Every `slither-disable` in the source carries a written justification next to it.

🏆 Why this fits both tracks

⚡ **Agentic Economy.** Not one agent, a **market**. Five agents with onchain identities trade across three rails: ERC-8183 escrow holds job fees, x402 sells the judgement, direct USDC moves the bill money. The evaluator reads the chain and can refuse to pay the provider. And seven endpoints are **open for any stranger's agent to buy**.

🏛️ **DeFi.** A working treasury over escrowed onchain debt: cross-bill net-position aggregation, **atomic batched settlement**, every amount re-read from chain before signing, all-or-nothing escrow with self-service refunds, and no custody anywhere in the path.

> One product. Real USDC on Arc. Agents that pay their own way, and one that gets paid to say no.

Splitsy

🙏 Thank you

splitsy.xyz

github.com/qFloppa/Splitsy

Arc · Circle Wallets · CCTP v2 · Gateway · Paymaster · x402 · ERC-8004 · ERC-8183

Programmable Money Hackathon